

Atividade: Consolidação dos Princípios SOLID

Disciplina: Design de Software

Tópico: Refatoração Arquitetural e Princípios SOLID (SRP, OCP, LSP, ISP, DIP)

1. Objetivo da Atividade

A presente atividade tem como objetivo avaliar a capacidade analítica e prática na identificação de anomalias arquiteturais (code smells) e na aplicação das cinco métricas qualitativas do acrônimo SOLID em um sistema orientado a objetos.

2. Contextualização do Cenário

O sistema em análise é um "Módulo de Gestão de Contratos e Faturamento" corporativo. O código-fonte atual foi desenvolvido sem o devido planejamento estrutural, resultando em um alto grau de acoplamento, baixa coesão e severas violações de contratos comportamentais.

O sistema atualmente sofre com fragilidade na manutenção: a adição de novos tipos de contratos exige a modificação de classes fundamentais, e o reaproveitamento de código é inviabilizado pela rigidez das dependências e interfaces.

3. Instruções para a Execução

A atividade consiste na refatoração completa dos artefatos de código fornecidos abaixo. Deverá ser entregue um repositório ou arquivo (.zip, .rar) contendo a nova estrutura do sistema seguindo os princípios SOLID abordados nas aulas.

4. Artefatos de Código Legado (Para Refatoração)

Artefato 1: A Interface Genérica

Java

```
public interface OperacoesSistema {  
    void processarPagamento(double valor);  
    void gerarNotaFiscal(String dadosContrato);  
}
```

```
void notificarCliente(String email, String mensagem);  
}
```

Artefato 2: O Componente de Pagamento

Java

```
public class ServicoPagamentoCartao implements OperacoesSistema {  
  
    @Override  
    public void processarPagamento(double valor) {  
        System.out.println("Processando pagamento via API de Cartão de Crédito: $" + valor);  
    }  
  
    @Override  
    public void gerarNotaFiscal(String dadosContrato) {  
        throw new UnsupportedOperationException("O serviço de cartão não emite nota fiscal.");  
    }  
  
    @Override  
    public void notificarCliente(String email, String mensagem) {  
        throw new UnsupportedOperationException("O serviço de cartão não envia e-mails.");  
    }  
}
```

Artefato 3: A Hierarquia de Domínio

Java

```
public class Contrato {  
    protected String titular;  
    protected double valorBase;  
  
    public Contrato(String titular, double valorBase) {  
        this.titular = titular;  
        this.valorBase = valorBase;  
    }  
}
```

```

    }

    public void assinar() {
        System.out.println("Contrato assinado por: " + titular);
    }

    public void renovar() {
        System.out.println("Renovando o contrato por mais 12 meses.");
        this.valorBase += (this.valorBase * 0.10);
    }

    public double getValorBase() { return valorBase; }
    public String getTitular() { return titular; }
}

public class ContratoVitalicio extends Contrato {

    public ContratoVitalicio(String titular, double valorBase) {
        super(titular, valorBase);
    }

    @Override
    public void renovar() {
        throw new IllegalStateException("Contratos vitalícios não possuem prazo de validade e não podem ser renovados.");
    }
}

```

Artefato 4: O Processador Central (God Class)

Java

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class GerenciadorDeContratos {

```

```

private ServicoPagamentoCartao servicoPagamento;

public GerenciadorDeContratos() {
    this.servicoPagamento = new ServicoPagamentoCartao();
}

public void processarEfetivacao(Contrato contrato, String modalidadePagamento, String
emailCliente) {

    double valorFinal = contrato.getValorBase();

    if (modalidadePagamento.equalsIgnoreCase("A_VISTA")) {
        valorFinal = valorFinal * 0.90;
    } else if (modalidadePagamento.equalsIgnoreCase("PARCELADO_CARTAO")) {
        valorFinal = valorFinal * 1.05;
    } else if (modalidadePagamento.equalsIgnoreCase("BOLETO")) {
        valorFinal = valorFinal + 5.00;
    } else {
        throw new IllegalArgumentException("Modalidade de pagamento inválida.");
    }

    servicoPagamento.processarPagamento(valorFinal);

    try {
        Connection conn = DriverManager.getConnection("jdbc:postgresql://localhost:5432/erp",
"admin", "1234");
        String sql = "INSERT INTO contratos_ativos (titular, valor_final) VALUES (?, ?)";
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setString(1, contrato.getTitular());
        stmt.setDouble(2, valorFinal);
        stmt.executeUpdate();
        conn.close();
    } catch (Exception e) {
        System.err.println("Erro ao salvar o contrato no banco de dados.");
    }

    System.out.println("Conectando ao servidor SMTP local...");
    System.out.println("Enviando e-mail para: " + emailCliente);
    System.out.println("Mensagem: Seu contrato foi efetivado com sucesso. Valor: " + valorFinal);
}
}

```

